



EC200U&EG91xU Series QuecOpen(SDK)

Low Power Consumption

API Reference Manual

LTE Standard Module Series

Version: 1.1

Date: 2023-10-10

Status: Released



At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.

Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China

Tel: +86 21 5108 6236

Email: info@quectel.com

Or our local offices. For more information, please visit:

<http://www.quectel.com/support/sales.htm>.

For technical support, or to report documentation errors, please visit:

<http://www.quectel.com/support/technical.htm>.

Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

- a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
- b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
- c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
- d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

Copyright © Quectel Wireless Solutions Co., Ltd. 2023. All rights reserved.

About the Document

Revision History

Version	Date	Author	Description
-	2021-10-08	Kevin WANG	Creation of the document
1.0	2021-10-21	Kevin WANG	First official release
1.1	2023-10-10	Felix Lin	1. Updated the document name based on the unified QuecOpen naming. 2. Added the applicable modules EG915U series and EG912U-GL. 3. Added the APIs to set and get the time for UART to enter into sleep after no data interaction (Chapters 2.3.9 and 2.3.10).

Contents

About the Document.....	3
Contents	4
Table Index.....	5
1 Introduction	6
1.1. Applicable Modules	6
2 Low Power Consumption API.....	7
2.1. Header File.....	7
2.2. API Overview	7
2.3. API Description	8
2.3.1. ql_autosleep_enable	8
2.3.1.1. ql_errcode_sleep.....	8
2.3.1.2. QL_SLEEP_FLAG_E	9
2.3.2. ql_autosleepex_enable	9
2.3.3. ql_lpm_wakelock_create.....	10
2.3.4. ql_lpm_wakelock_delete.....	10
2.3.5. ql_lpm_wakelock_lock	11
2.3.6. ql_lpm_wakelock_unlock	11
2.3.7. ql_sleep_register_cb.....	12
2.3.8. ql_wakeup_register_cb	12
2.3.9. ql_uart_set_sleep_delay_time	13
2.3.9.1. ql_uart_errcode_e	13
2.3.10. ql_uart_get_sleep_delay_time	14
3 Demo and Debugging.....	15
3.1. Demo Introduction.....	15
3.2. Low Power Consumption Debugging	15
3.2.1. Sleep Failure Analysis.....	15
3.2.2. Matters Needing Attention for Sleep Mode.....	16
4 Appendix References	17

Table Index

Table 1: Applicable Modules 6

Table 2: API Overview 7

Table 3: Related Document 17

Table 4: Terms and Abbreviations 17

1 Introduction

Quectel EC200U series and EG91xU family modules support QuecOpen® solution. QuecOpen is an embedded development platform based on RTOS, which is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see **document [1]**.

This document is applicable to QuecOpen® solution based on CSDK build environment. It mainly introduces the API functions and debugging methods related to low power consumption function of EC200U series and EG91xU family modules.

1.1. Applicable Modules

Table 1: Applicable Modules

Module Family	Module
-	EC200U Series
EG91xU	EG912U-GL
	EG915U Series

2 Low Power Consumption API

2.1. Header File

ql_power.h is the header file of low power consumption API, and *ql_uart.h* is the header file of UART sleep API without data interaction. They are all located in the directory of *ql-sdk\ql-components\ql-kernel\inc*. Unless otherwise specified, all header files mentioned in this document are in this directory.

2.2. API Overview

Table 2: API Overview

Function	Description
<i>ql_autosleep_enable()</i>	Enables/disables the system to sleep.
<i>ql_autosleepex_enable()</i>	Enables enhanced sleep.
<i>ql_lpm_wakelock_create()</i>	Creates a wake lock.
<i>ql_lpm_wakelock_delete()</i>	Deletes the created wake lock.
<i>ql_lpm_wakelock_lock()</i>	Locks the wake lock.
<i>ql_lpm_wakelock_unlock()</i>	Unlocks the wake lock.
<i>ql_sleep_register_cb()</i>	Registers a sleep callback function.
<i>ql_wakeup_register_cb()</i>	Registers a wake callback function.
<i>ql_uart_set_sleep_delay_time()</i>	Sets the time for UART to enter into sleep after no data interaction.
<i>ql_uart_get_sleep_delay_time()</i>	Gets the time for UART to enter into sleep after no data interaction.

2.3. API Description

2.3.1. ql_autosleep_enable

This function enables/disables the system to sleep.

- **Prototype**

```
ql_errcode_sleep ql_autosleep_enable(QL_SLEEP_FLAG_E sleep_flag)
```

- **Parameter**

sleep_flag:

[In] The flag of enabling or disabling the system to sleep. See **Chapter 2.3.1.2** for details.

- **Return Value**

See **Chapter 2.3.1.1** for error codes.

2.3.1.1. ql_errcode_sleep

```
typedef enum
{
    QL_SLEEP_SUCCESS                = QL_SUCCESS,
    QL_SLEEP_INVALID_PARAM          = (QL_COMPONENT_PM_SLEEP << 16) | 1000,
    QL_SLEEP_LOCK_CREATE_FAIL       = (QL_COMPONENT_PM_SLEEP << 16) | 1001,
    QL_SLEEP_LOCK_DELETE_FAIL       = (QL_COMPONENT_PM_SLEEP << 16) | 1002,
    QL_SLEEP_LOCK_LOCK_FAIL         = (QL_COMPONENT_PM_SLEEP << 16) | 1003,
    QL_SLEEP_LOCK_UNLOCK_FAIL       = (QL_COMPONENT_PM_SLEEP << 16) | 1004,
    QL_SLEEP_LOCK_AUTOSLEEP_FAIL    = (QL_COMPONENT_PM_SLEEP << 16) | 1005,
    QL_SLEEP_PARAM_SAVE_FAIL        = (QL_COMPONENT_PM_SLEEP << 16) | 1006,
}ql_errcode_sleep
```

- **Member**

Member	Description
<i>QL_SLEEP_SUCCESS</i>	Successful execution
<i>QL_SLEEP_INVALID_PARAM</i>	Invalid parameter
<i>QL_SLEEP_LOCK_CREATE_FAI</i>	Failed to create a sleep lock

<code>QL_SLEEP_LOCK_DELETE_FAIL</code>	Failed to delete a sleep lock
<code>QL_SLEEP_LOCK_LOCK_FAIL</code>	Failed to lock a sleep lock
<code>QL_SLEEP_LOCK_UNLOCK_FAIL</code>	Failed to unlock a sleep lock
<code>QL_SLEEP_LOCK_AUTOSLEEP_FAIL</code>	Failed to start automatic sleep
<code>QL_SLEEP_PARAM_SAVE_FAIL</code>	Failed to save the parameter

2.3.1.2. QL_SLEEP_FLAG_E

```
typedef enum
{
    QL_NOT_ALLOW_SLEEP = 0,
    QL_ALLOW_SLEEP,
}QL_SLEEP_FLAG_E
```

- **Member**

Member	Description
<code>QL_NOT_ALLOW_SLEEP</code>	Disable the system to sleep
<code>QL_ALLOW_SLEEP</code>	Enable the system to sleep

2.3.2. ql_autosleepex_enable

This function enables enhanced sleep.

- **Prototype**

```
ql_errcode_sleep ql_autosleepex_enable(QL_SLEEP_FLAG_E sleep_flag, uint8_t no_data_time,
uint16_t punish_time)
```

- **Parameter**

sleep_flag:

[In] The flag of enabling or disabling the system to sleep. See **Chapter 2.3.1.2** for details.

no_data_time:

[In] The duration of releasing RRC after no data interaction. Unit: s.

punish_time:

[In] The duration of restoring the enhanced sleep mode if an abnormal network registration is encountered.
Unit: s.

- **Return Value**

See **Chapter 2.3.1.1** for error codes.

2.3.3. ql_lpm_wakelock_create

This function creates a wake lock.

- **Prototype**

```
int ql_lpm_wakelock_create(char *lock_name, int name_size)
```

- **Parameter**

lock_name:

[In] The name of a wake lock. The maximum length is 32 bytes.

name_size:

[In] Invalid parameter.

- **Return Value**

Less than or equal to 0	Failed execution
More than 0	Successful execution. Returns the handle of the lock, which is passed in by subsequent operations.

2.3.4. ql_lpm_wakelock_delete

This function deletes the created wake lock.

- **Prototype**

```
ql_errcode_sleep ql_lpm_wakelock_delete(int wakelock_fd)
```

- **Parameter**

wakelock_fd:

[In] Handle of the wake lock. Returned by *ql_lpm_wakelock_create()*.

- **Return Value**

See **Chapter 2.3.1.1** for error codes.

2.3.5. `ql_lpm_wakelock_lock`

This function locks the wake lock. If there are one or more wake locks in the system, the system is not allowed to sleep.

- **Prototype**

```
ql_errcode_sleep ql_lpm_wakelock_lock(int wakelock_fd)
```

- **Parameter**

wakelock_fd:

[In] Handle of the wake lock. Returned by *ql_lpm_wakelock_create()*.

- **Return Value**

See **Chapter 2.3.1.1** for error codes.

2.3.6. `ql_lpm_wakelock_unlock`

This function unlocks the wake lock. Only when all wake locks in the system are unlocked can the system sleep.

- **Prototype**

```
ql_errcode_sleep ql_lpm_wakelock_unlock(int wakelock_fd)
```

- **Parameter**

wakelock_fd:

[In] Handle of the wake lock. Returned by *ql_lpm_wakelock_create()*.

- **Return Value**

See **Chapter 2.3.1.1** for error codes.

2.3.7. ql_sleep_register_cb

This function registers a sleep callback function.

- **Prototype**

```
ql_errcode_sleep ql_sleep_register_cb(ql_enter_sleep_callback cb)
```

- **Parameter**

cb:

[In] Sleep callback function pointer.

- **Return Value**

See **Chapter 2.3.1.1** for error codes.

2.3.8. ql_wakeup_register_cb

This function registers a wakeup callback function.

- **Prototype**

```
ql_errcode_sleep ql_wakeup_register_cb(ql_exit_sleep_callback cb)
```

- **Parameter**

cb:

[In] Wakeup callback function pointer.

- **Return Value**

See **Chapter 2.3.1.1** for error codes.

2.3.9. ql_uart_set_sleep_delay_time

This function sets the time for UART to enter into sleep after no data interaction.

● Prototype

```
ql_uart_errcode_e ql_uart_set_sleep_delay_time(uint8_t times)
```

● Parameter

times:

[In] The time for UART to enter into sleep after no data interaction. Unit: s. The minimum value: 1; the maximum value: 255.

● Return Value

See **Chapter 2.3.9.1** for error codes.

2.3.9.1. ql_uart_errcode_e

The return value of UART sleep API without data interaction indicates whether the function is executed successfully. If not, the error reason is returned. The enumeration of UART error codes is defined as below:

```
typedef enum
{
    QL_UART_SUCCESS                = 0,
    QL_UART_EXECUTE_ERR            = 1|QL_UART_ERRCODE_BASE,
    QL_UART_MEM_ADDR_NULL_ERR,
    QL_UART_INVALID_PARAM_ERR,
    QL_UART_OPEN_REPEAT_ERR,
    QL_UART_NOT_OPEN_ERR          = 5|QL_UART_ERRCODE_BASE,
} ql_uart_errcode_e
```

● Member

Member	Description
QL_UART_SUCCESS	Successful execution
QL_UART_EXECUTE_ERR	Failed execution
QL_UART_MEM_ADDR_NULL_ERR	The parameter address is NULL
QL_UART_INVALID_PARAM_ERR	Invalid parameter

<code>QL_UART_OPEN_REPEAT_ERR</code>	UART repeatedly opened
<code>QL_UART_NOT_OPEN_ERR</code>	UART is not open

2.3.10. ql_uart_get_sleep_delay_time

This function gets the time for UART to enter into sleep after no data interaction.

- **Prototype**

```
ql_uart_errcode_e ql_uart_get_sleep_delay_time(uint8_t *times)
```

- **Parameter**

times:

[Out] The time for UART to enter into sleep after no data interaction. Unit: s.

- **Return Value**

See **Chapter 2.3.9.1** for error codes.

3 Demo and Debugging

This chapter mainly introduces how to use the above API functions in Kernel and Application, and debug the low power consumption function.

3.1. Demo Introduction

power_demo.c, the Application demo, is located in the directory of `components\ql-application\power` of CSDK. Two sleep locks and a timer are created with power task in the demo code. After the timer runs for 1 second, these two wake locks will be released and the system is enabled to sleep.

3.2. Low Power Consumption Debugging

3.2.1. Sleep Failure Analysis

1. Only when all conditions below are met can the system sleep:
 - USB device is plugged out.
 - All system wake locks are unlocked.
 - No external pin interruption or interference.
 - The system is enabled to automatic sleep.
 - **AT+CFUN=0** is set or the system is in a normal network registration state.
2. Capture the log for failing to enter into sleep mode.

If all the conditions mentioned above are met but the system still cannot sleep for a long time, the log indicating the reason why the system cannot sleep will be printed out every 3 seconds, as shown in the figure below. Please save the log and submit it to Quectel Technical Supports for further analysis.

13:59:18.823	15880	QUEC/I	[q1_POWER][q1_lpm_wakelock_unlock, 337] unlock ok
13:59:18.823	15882	QUEC/I	[q1_POWER][q1_lpm_wakelock_unlock, 337] unlock ok
13:59:18.823	15882	QUEC/I	[q1_POWER][q1_lpm_wakelock_unlock, 337] unlock ok
13:59:18.823	15882	QUEC/I	[q1_POWER][q1_lpm_wakelock_unlock, 337] unlock ok
13:59:21.810	65035	QUEC/I	[q1_POWER][q1_lpm_wakelock_unlock, 337] unlock ok
13:59:21.831	65035	QUEC/I	[q1_POWER][q1_lpm_wakelock_unlock, 337] unlock ok
13:59:24.799	48650	QUEC/I	[q1_POWER][q1_lpm_wakelock_unlock, 337] unlock ok
13:59:24.819	48651	QUEC/I	[q1_POWER][q1_lpm_wakelock_unlock, 337] unlock ok
13:59:27.798	32267	QUEC/I	[q1_POWER][q1_lpm_wakelock_unlock, 337] unlock ok
13:59:27.860	32267	QUEC/I	[q1_POWER][q1_lpm_wakelock_unlock, 337] unlock ok
13:59:30.813	15883	QUEC/I	[q1_POWER][q1_lpm_wakelock_unlock, 337] unlock ok
13:59:30.813	15883	QUEC/I	[q1_POWER][q1_lpm_wakelock_unlock, 337] unlock ok

3.2.2. Matters Needing Attention for Sleep Mode

In sleep mode, if the flow control of main UART is not enabled, data more than 127 bytes cannot be sent directly to the module through the main UART, and can only be sent after exiting sleep mode, otherwise it may be lost.

4 Appendix References

Table 3: Related Document

Document Name
[1] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Quick_Start_Guide

Table 4: Terms and Abbreviations

Abbreviation	Description
API	Application Programming Interface
CSDK	Customized Software Development Kit
IoT	Internet of Things
RRC	Radio Resource Control
RTOS	Real-Time Operating System
USB	Universal Serial Bus